Distributed Systems: Zero to Hero

Oct 19, 2026 · @Subhamoy ghosh

1. Foundations: why systems get distributed

A distributed system is several machines cooperating over an unreliable network so they look like one system; you build one to scale past a single box, survive failures, and decouple teams. The price is that the network can drop, delay, duplicate and reorder messages, and any machine can die mid-task.

The core idea: put a buffer between producer and consumer

Every tool in this document (queues, Kafka, RabbitMQ, Redis, SNS/SQS) is a variation on one move: **don't call the other service directly; drop a message into a durable buffer and let it be processed later.** This gives you:

- **Decoupling:** producer doesn't know who consumes, or whether they are up.
- **Load levelling:** a spike of 10,000 requests/s is absorbed by the buffer and drained at the speed workers can sustain.
- **Horizontal scaling:** add workers to drain faster.
- **Fault isolation:** a crashed consumer does not crash the producer; messages wait.

Vocabulary

Term	Meaning
Producer / Publisher	Sends messages
Consumer / Worker / Subscriber	Receives and processes messages
Broker	Server that stores and routes messages (RabbitMQ, Kafka, Redis, SQS)
Queue	One message goes to **one** consumer (work distribution)
Topic / Pub-Sub	One message goes to **every** interested subscriber (broadcast)
Fanout	One event triggers many independent downstream actions
Ack	Consumer tells the broker "done, delete it"

Term	Meaning
DLQ	Dead-letter queue: where poison messages go after too many failures
Offset	Kafka's bookmark of how far a consumer has read in a log

Delivery guarantees

Guarantee	What it means	How it happens
At-most-once	Message may be lost, never duplicated	Auto-ack before processing
At-least-once	Never lost, may be duplicated	Ack **after** processing (the default you should assume)
Exactly-once	Processed once in effect	Only achievable as at-least-once **plus** idempotent consumers (or Kafka transactions inside Kafka)

The network makes true exactly-once delivery impossible in general (the Two Generals problem), so the working rule is: **assume duplicates, make handlers idempotent.**

Idempotency in JavaScript

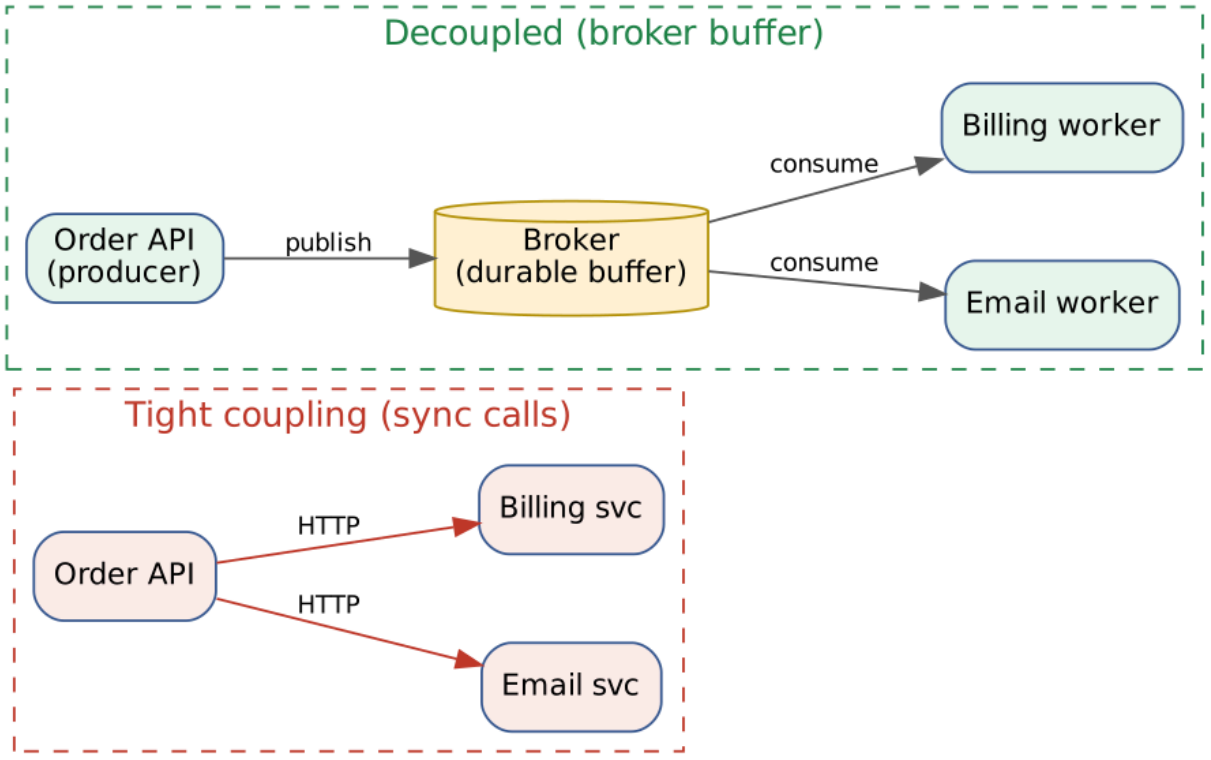
An idempotent handler produces the same result when run twice with the same message. The usual trick is a unique message ID checked before doing the work:

```
// Idempotent consumer: dedupe by message id (use Redis SET NX or a DB unique
key in production)
const processed = new Set();

async function handle(msg) {
  if (processed.has(msg.id)) {
    console.log('duplicate, skipping', msg.id);
    return;
  }
  await chargeCustomer(msg.orderId, msg.amount); // side effect
  processed.add(msg.id);                          // in prod: same DB
transaction as the side effect
}
```

CAP, ordering and consistency

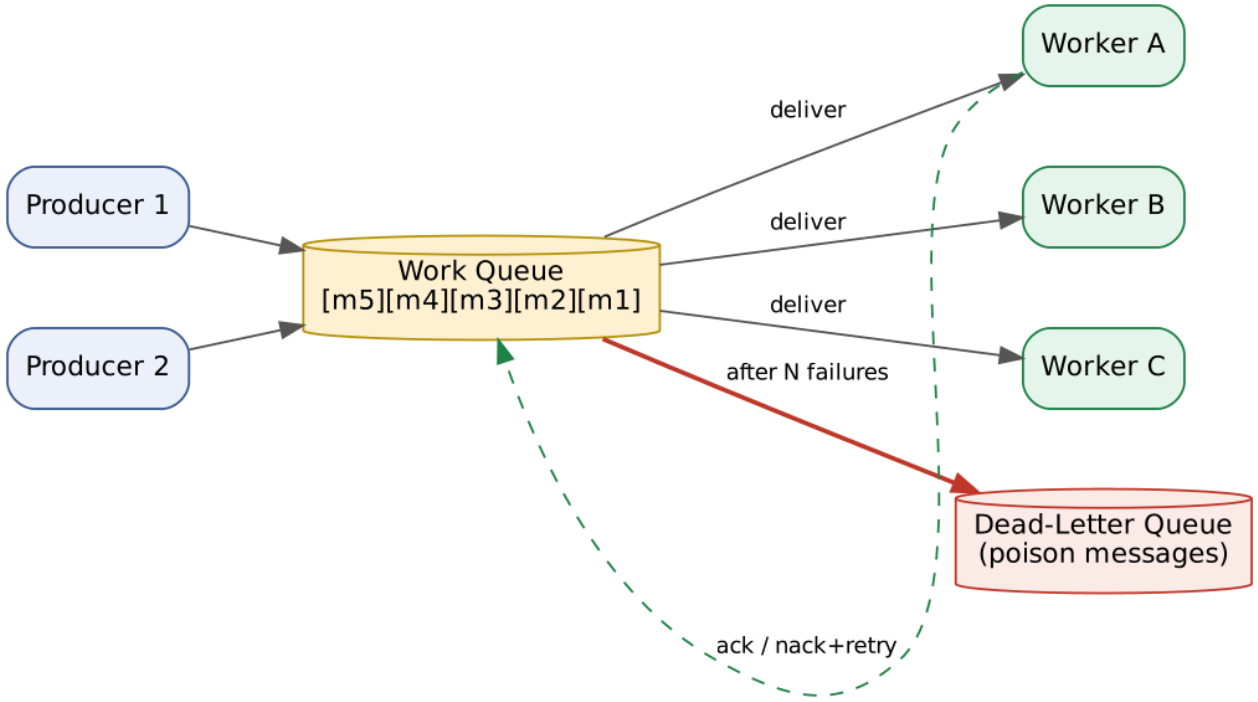
- **CAP theorem:** during a network partition you must choose Consistency (refuse requests that might be stale) or Availability (answer, possibly stale). Kafka and RabbitMQ quorum queues lean CP; Redis pub/sub and SQS standard lean AP.
- **Ordering:** global ordering does not scale. Kafka guarantees order **within a partition**; SQS FIFO guarantees order **within a message group**. You pick a key (e.g. `userId`) so related messages stay in order.
- **Eventual consistency:** after a write, other services converge over milliseconds to seconds, not instantly.
- **Backpressure:** when consumers are slower than producers, queues grow; you need limits, prefetch caps and autoscaling.



Read it left to right: on top, the Order API calls each service directly and breaks if one is down; below, it publishes once to a broker and each worker pulls at its own pace.

2. Queues and workers

A queue delivers each message to exactly one consumer, so adding workers multiplies throughput; this is the **competing consumers** pattern and the workhorse of background processing (emails, image resizing, PDF generation, payments, webhooks).



The life of a message

1. Producer sends a message; the broker persists it.
2. A worker receives it. The broker hides it (**visibility timeout** in SQS, **unacked** state in RabbitMQ) rather than deleting it.
3. Worker finishes and **acks**; the broker deletes it. If the worker crashes or times out, the message reappears for another worker.
4. After N failed attempts the broker moves it to a **dead-letter queue (DLQ)** so one poison message cannot block the line forever.

Design rules that matter in production

- **Ack after success, never before.** That is what gives at-least-once delivery.
- **Handlers must be idempotent** (section 1), because crashes between "work done" and "ack sent" cause redelivery.
- **Retry with exponential backoff and jitter**, so a failing dependency is not hammered by synchronized retries.
- **Limit concurrency / prefetch** so a worker does not grab 1,000 messages it cannot finish before the visibility timeout.
- **Alert on queue depth and age of oldest message**; those two numbers tell you when workers are falling behind. Autoscale workers on them.
- **Shut down gracefully:** on SIGTERM stop receiving, finish in-flight jobs, then exit.
- **Watch the DLQ.** An unwatched DLQ is silent data loss.

Code 1: a queue from scratch (runs as is with Node 18+, no dependencies)

This mimics SQS semantics, so you can see visibility, ack, retry and DLQ with your own eyes. Save as `queue.mjs` and run `node queue.mjs`.

```
// A tiny SQS-like queue: visibility timeout, ack/nack, retries, dead-letter
queue.
class Queue {
  constructor({ visibilityMs = 500, maxReceives = 3 } = {}) {
    this.ready = [];            // waiting messages
    this.inflight = new Map();  // delivered, not yet acked
    this.dlq = [];              // poison messages end up here
    this.visibilityMs = visibilityMs;
    this.maxReceives = maxReceives;
    this.seq = 0;
  }
  send(body) { this.ready.push({ id: ++this.seq, body, receives: 0 }); }
  receive() {
    const msg = this.ready.shift();
    if (!msg) return null;
    msg.receives++;
    // If the worker crashes and never acks, the message reappears after the
timeout.
    const timer = setTimeout(() => this.#requeue(msg.id), this.visibilityMs);
    this.inflight.set(msg.id, { msg, timer });
    return msg;
  }
  ack(id)  { const e = this.inflight.get(id); if (e) { clearTimeout(e.timer);
this.inflight.delete(id); } }
  nack(id) { const e = this.inflight.get(id); if (e) { clearTimeout(e.timer);
this.#requeue(id); } }
  #requeue(id) {
    const e = this.inflight.get(id);
    if (!e) return;
    this.inflight.delete(id);
    (e.msg.receives >= this.maxReceives ? this.dlq : this.ready).push(e.msg);
  }
  get idle() { return !this.ready.length && !this.inflight.size; }
}

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

async function worker(name, q, handler) {
  while (true) {
    const msg = q.receive();
    if (!msg) { if (q.idle) return; await sleep(20); continue; }
```

```
    try {
      await handler(msg.body);
      q.ack(msg.id);                                  // ack AFTER success
=> at-least-once
      console.log(`[${name}] done  #${msg.id} ${msg.body.job}`);
    } catch (err) {
      console.log(`[${name}] fail  #${msg.id} ${msg.body.job} (attempt
${msg.receives})`);
      q.nack(msg.id);
    }
  }
}

const q = new Queue({ maxReceives: 3 });
['resize-1', 'resize-2', 'bad-image', 'resize-3', 'resize-4'].forEach((job)
=> q.send({ job }));

const handler = async ({ job }) => {
  await sleep(50);
  if (job.startsWith('bad')) throw new Error('cannot decode');
};

// 3 workers compete for the same queue => horizontal scaling
await Promise.all(['A', 'B', 'C'].map((n) => worker(n, q, handler)));
console.log('DLQ:', q.dlq.map((m) => m.body.job));
```

Output from an actual run: jobs 1, 2, 4, 5 finish across workers A, B and C; `bad-image` fails three times and ends with `DLQ: [ 'bad-image' ]`.

Code 2: a crash-safe queue on Redis lists

`BLMOVE` atomically moves a job from `ready` to `processing`, so a worker crash never loses it; a separate reaper can push stale `processing` items back.

```
import Redis from 'ioredis';
import { randomUUID } from 'node:crypto';

const redis = new Redis();            // producer connection
const blocker = new Redis();          // blocking commands need their own
connection

export const enqueue = (type, data) =>
  redis.lpush('jobs:ready', JSON.stringify({ id: randomUUID(), type, data,
attempts: 0 }));

export async function runWorker(handle) {
```

```
  while (true) {
    // pop oldest from ready, push onto processing, wait up to 5s
    const raw = await blocker.blmove('jobs:ready', 'jobs:processing',
'RIGHT', 'LEFT', 5);
    if (!raw) continue;
    const job = JSON.parse(raw);
    try {
      await handle(job);
      await redis.lrem('jobs:processing', 1, raw);          // ack
    } catch (err) {
      await redis.lrem('jobs:processing', 1, raw);
      job.attempts += 1;
      const target = job.attempts >= 3 ? 'jobs:dead' : 'jobs:ready';  //
retry or DLQ
      await redis.lpush(target, JSON.stringify(job));
    }
  }
}
```

Code 3: BullMQ, the production-grade way on Redis

BullMQ adds retries with backoff, delayed jobs, priorities, rate limits, repeatable jobs and a dashboard ecosystem.

```
import { Queue, Worker } from 'bullmq';

const connection = { host: '127.0.0.1', port: 6379 };
const emailQueue = new Queue('email', { connection });

// Producer
await emailQueue.add('welcome', { userId: 42 }, {
  attempts: 5,
  backoff: { type: 'exponential', delay: 1000 },   // 1s, 2s, 4s, 8s...
  removeOnComplete: 1000,
  delay: 0,                                         // set e.g. 60_000 for a
delayed job
  priority: 1,
});

// Worker (run many copies, on many machines)
const worker = new Worker('email', async (job) => {
  await sendWelcomeEmail(job.data.userId);
}, { connection, concurrency: 10 });

worker.on('failed', (job, err) => console.error('failed', job?.id,
err.message));
```

```
// Graceful shutdown
process.on('SIGTERM', async () => { await worker.close(); process.exit(0);
});
```

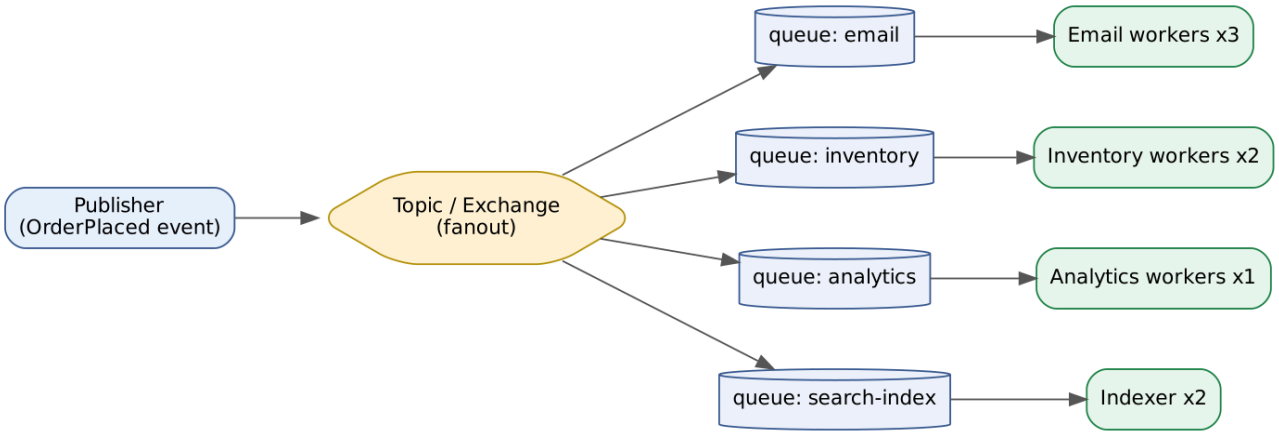
A plain backoff helper, if you build retries yourself:

```
const backoffMs = (attempt) => Math.min(30_000, 2 ** attempt * 100) * (0.5 +
Math.random() / 2); // jitter
```

Code 2 and 3 need a running Redis and `npm i ioredis bullmq`; they were not run against a live server in this document's build.

3. Pub/Sub and fanout architecture

Pub/sub delivers each message to **every** subscriber, and fanout is the architecture built on it: one event ("OrderPlaced") triggers many independent reactions without the publisher knowing any of them exist. Adding a new reaction means adding a subscriber, not editing the order service.



Each subscriber gets its OWN copy; inside one subscriber, workers compete

The key picture: **fanout between subscribers, competition inside each subscriber.** Each subscriber owns a queue (so its failures and slowness are isolated), and that queue is drained by a pool of competing workers.

Queue vs pub/sub vs log

	Work queue	Pub/Sub topic	Log (Kafka)
Who gets a message	One consumer	All subscribers	Every consumer **group**, one consumer within a group

	Work queue	Pub/Sub topic	Log (Kafka)
Message after consumption	Deleted	Deleted (or never stored)	Kept until retention expires
Replay old messages	No	No	Yes, by resetting the offset
Typical use	Jobs	Notifications, broadcast	Event streaming, audit, analytics

Fanout patterns you will meet

- **Topic + queue per subscriber** (SNS to SQS, RabbitMQ fanout exchange): the most robust form, because every consumer has its own buffer and DLQ.
- **Ephemeral pub/sub** (Redis `PUBLISH`, WebSocket rooms): subscribers must be connected at that moment or they miss the message. Fine for cache invalidation and live UI updates, wrong for anything you cannot lose.
- **Content-based filtering:** subscribers declare interest (RabbitMQ topic exchange `eu.orders.*`, SNS filter policies, EventBridge rules) so they receive only matching events.
- **Fanout-on-write vs fanout-on-read:** a social feed can copy a post into every follower's timeline at write time (fast reads, expensive for celebrities with 50M followers) or assemble timelines at read time (cheap writes, slow reads). Large systems mix both: write-fanout for normal users, read-merge for celebrities.
- **Event notification vs event-carried state:** a thin event (`{orderId}`) makes subscribers call back for data; a fat event carries the data, removing the callback but making schema changes more delicate.

Rules for good events

- Name them in the past tense, as facts: `OrderPlaced`, not `PlaceOrder`.
- Include `eventId`, `type`, `occurredAt`, a `version` and a correlation ID.
- Treat the schema as a public API; add fields, don't rename them.
- Make every subscriber idempotent; fanout multiplies redelivery risk.

Code 4: fanout with isolated subscribers (runs as is, no dependencies)

Save as `fanout.mjs`. Email has 2 workers, inventory 1, and analytics is deliberately broken to prove isolation.

```
// In-memory topic with fanout: every subscriber gets its OWN copy and its
OWN queue.
class Topic {
  constructor(name) { this.name = name; this.subs = new Map(); }

  subscribe(name, handler, { workers = 1 } = {}) {
    this.subs.set(name, { queue: [], handler, running: 0, workers });
  }

  publish(event) {
    for (const [name, sub] of this.subs) {
      sub.queue.push(structuredClone(event));   // separate copy per
subscriber
      this.#drain(name, sub);
    }
  }

  #drain(name, sub) {
    while (sub.running < sub.workers && sub.queue.length) {
      const ev = sub.queue.shift();
      sub.running++;
      Promise.resolve()
        .then(() => sub.handler(ev))
        .catch((e) => console.log(`  [${name}] FAILED ${ev.type}
#${ev.orderId}: ${e.message}`))
        .finally(() => { sub.running--; this.#drain(name, sub); });
    }
  }
}

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const orders = new Topic('orders');

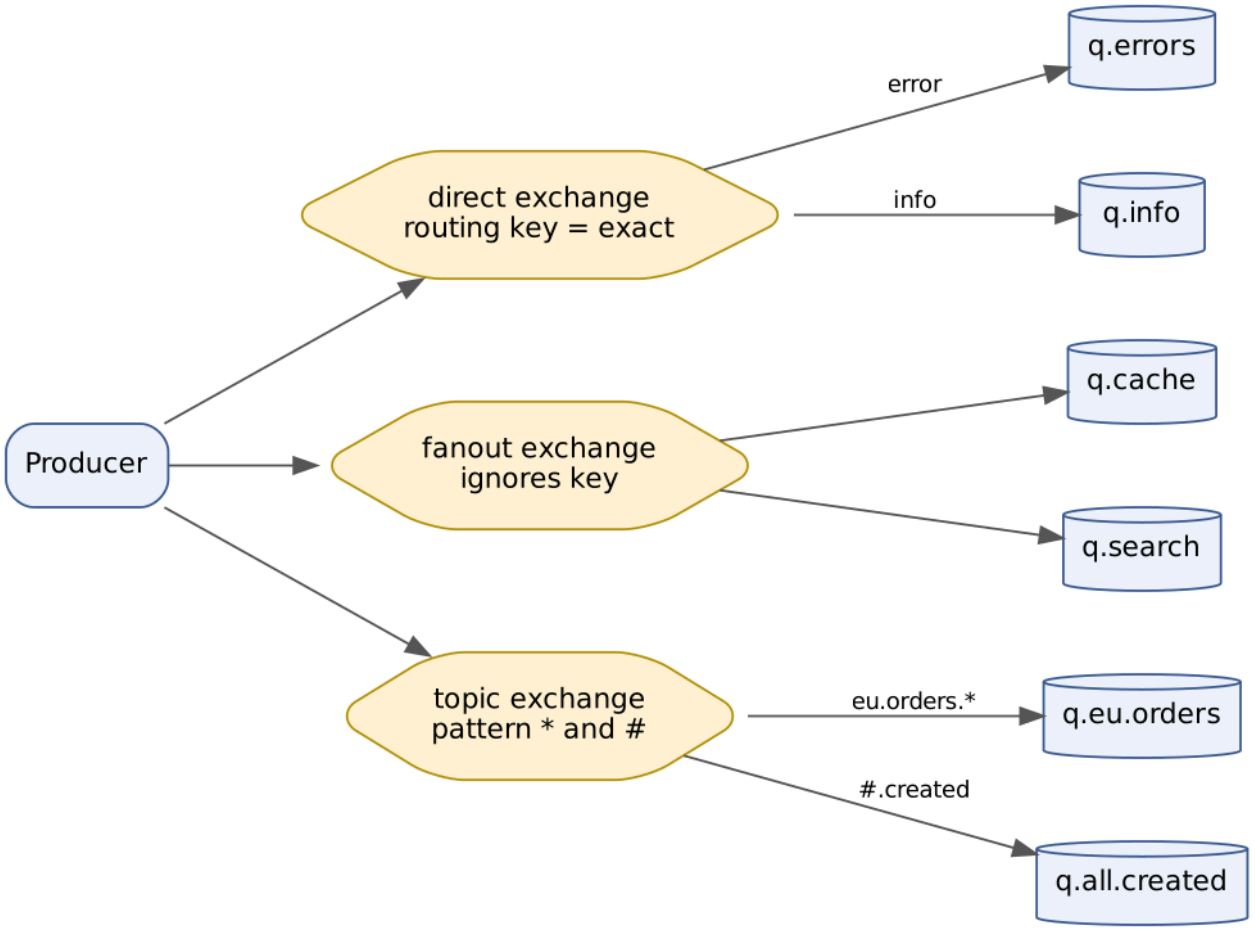
orders.subscribe('email',     async (e) => { await sleep(30); console.log(`
[email]     sent receipt for #${e.orderId}`); }, { workers: 2 });
orders.subscribe('inventory', async (e) => { await sleep(10); console.log(`
[inventory] reserved stock for #${e.orderId}`); });
orders.subscribe('analytics', async (e) => { throw new Error('warehouse
down'); });   // one broken subscriber

[101, 102, 103].forEach((id) => orders.publish({ type: 'OrderPlaced',
orderId: id }));
await sleep(300);
```

Actual output: analytics fails for all three orders, while email and inventory still process orders 101, 102 and 103. A broken subscriber does not hurt the others, which is exactly why production systems give each subscriber its own queue.

4. RabbitMQ: smart routing, classic queues

RabbitMQ is a broker built on AMQP whose strength is **routing**: producers never write to queues directly; they publish to an **exchange**, which routes copies to **queues** according to **bindings**. It covers work queues, pub/sub, filtered routing and RPC with one model, and it deletes messages once acknowledged.



Core concepts

Concept	Meaning
Connection / Channel	One TCP connection per app, many lightweight channels inside it (one channel per thread or task)

Concept	Meaning
Exchange	Receives messages and routes them. Types: **direct** (exact routing key), **fanout** (all bound queues), **topic** (patterns: `*` = one word, `#` = zero or more), **headers**
Queue	Buffer that holds messages for consumers. Make it `durable`; prefer **quorum queues** for replicated safety
Binding	Rule linking an exchange to a queue (with a key or pattern)
Ack / Nack	Consumer confirms success or failure; unacked messages are redelivered if the channel dies
Prefetch (QoS)	Max unacked messages per consumer; this is your backpressure knob
Publisher confirms	Broker tells the producer a message is safely stored
DLX (dead-letter exchange)	Where rejected, expired or overflowed messages are re-routed
TTL	Expiry per message or per queue; with a DLX it gives you delayed retry

Durability checklist (all three, or messages vanish on restart)

1. Queue declared `durable: true` (or a quorum queue).
2. Message published with `persistent: true`.
3. Publisher confirms enabled, and consumers ack manually.

Code 5: durable work queue with confirms, prefetch and manual ack

```
// npm i amqplib       (broker: docker run -p 5672:5672 -p 15672:15672
rabbitmq:3-management)
import amqp from 'amqplib';
import { randomUUID } from 'node:crypto';

const conn = await amqp.connect('amqp://localhost');

// ---------- producer ----------
const pub = await conn.createConfirmChannel();
```

```
await pub.assertQueue('tasks', { durable: true, arguments: { 'x-queue-type':
'quorum' } });

export async function enqueue(job) {
  pub.sendToQueue('tasks', Buffer.from(JSON.stringify(job)), {
    persistent: true,
    messageId: randomUUID(),          // use it for idempotency in the
consumer
    contentType: 'application/json',
  });
  await pub.waitForConfirms();         // resolves once the broker has it
safely
}

// ---------- worker (run many copies) ----------
const ch = await conn.createChannel();
await ch.assertQueue('tasks', { durable: true, arguments: { 'x-queue-type':
'quorum' } });
await ch.prefetch(5);                  // at most 5 unacked messages at a
time

await ch.consume('tasks', async (msg) => {
  if (!msg) return;
  try {
    await handle(JSON.parse(msg.content.toString()));
    ch.ack(msg);
  } catch (err) {
    ch.nack(msg, false, false);        // requeue=false => goes to the DLX if
configured
  }
});
```

Code 6: pub/sub with a topic exchange, one queue per service, and a DLX

```
import amqp from 'amqplib';

const conn = await amqp.connect('amqp://localhost');
const ch = await conn.createChannel();

await ch.assertExchange('events', 'topic', { durable: true });
await ch.assertExchange('events.dlx', 'fanout', { durable: true });
await ch.assertQueue('dead-letters', { durable: true });
await ch.bindQueue('dead-letters', 'events.dlx', '');

// Each service owns its queue and chooses what it hears
async function subscribe(queue, pattern, handler) {
```

```
  await ch.assertQueue(queue, {
    durable: true,
    arguments: { 'x-dead-letter-exchange': 'events.dlx' },
  });
  await ch.bindQueue(queue, 'events', pattern);
  await ch.consume(queue, async (msg) => {
    try { await handler(JSON.parse(msg.content.toString()),
msg.fields.routingKey); ch.ack(msg); }
    catch { ch.nack(msg, false, false); }
  });
}

await subscribe('email-service',     'order.*',        (e, key) =>
console.log('email', key, e));
await subscribe('analytics-service', '#',              (e, key) =>
console.log('analytics', key, e));
await subscribe('eu-billing',        'order.eu.created', (e, key) =>
console.log('billing', key, e));

// Publisher
ch.publish('events', 'order.eu.created', Buffer.from(JSON.stringify({
orderId: 7 })), { persistent: true });
```

This single publish reaches analytics (matches `#`) and eu-billing (exact match); `email-service` gets only keys with exactly two words such as `order.created`.

Code 7: delayed retry using TTL and a DLX loop

A failed job goes to a `retry` queue where it waits for 10 seconds, then expires and is dead-lettered back to the main queue.

```
await ch.assertExchange('jobs', 'direct', { durable: true });
await ch.assertQueue('jobs.main', { durable: true,
  arguments: { 'x-dead-letter-exchange': 'jobs.retry' } });          // nack
=> retry exchange
await ch.assertQueue('jobs.wait',  { durable: true,
  arguments: { 'x-message-ttl': 10_000, 'x-dead-letter-exchange': 'jobs' }
}); // expire => back to main
await ch.assertExchange('jobs.retry', 'fanout', { durable: true });
await ch.bindQueue('jobs.wait', 'jobs.retry', '');
await ch.bindQueue('jobs.main', 'jobs', 'jobs.main');   // dead-lettered msgs
keep their original routing key
// Producers use ch.sendToQueue('jobs.main', ...). In the consumer:
ch.nack(msg, false, false) => waits 10s => redelivered.
// Count attempts via msg.properties.headers['x-death'] and park the message
after N tries.
```

Code 8: RPC (request/reply) over queues

```
import { randomUUID } from 'node:crypto';

// Server: answers requests on 'rpc.square'
await ch.assertQueue('rpc.square');
await ch.consume('rpc.square', (msg) => {
  const n = JSON.parse(msg.content.toString()).n;
  ch.sendToQueue(msg.properties.replyTo, Buffer.from(JSON.stringify({ result:
n * n })),
    { correlationId: msg.properties.correlationId });
  ch.ack(msg);
});

// Client: private reply queue + correlation id
async function call(n) {
  const { queue: replyTo } = await ch.assertQueue('', { exclusive: true });
  const correlationId = randomUUID();
  return new Promise(async (resolve, reject) => {
    const timer = setTimeout(() => reject(new Error('rpc timeout')), 5000);
    await ch.consume(replyTo, (msg) => {
      if (msg.properties.correlationId === correlationId) {
clearTimeout(timer); resolve(JSON.parse(msg.content.toString()).result); }
    }, { noAck: true });
    ch.sendToQueue('rpc.square', Buffer.from(JSON.stringify({ n })), {
replyTo, correlationId });
  });
}
console.log(await call(12)); // 144
```

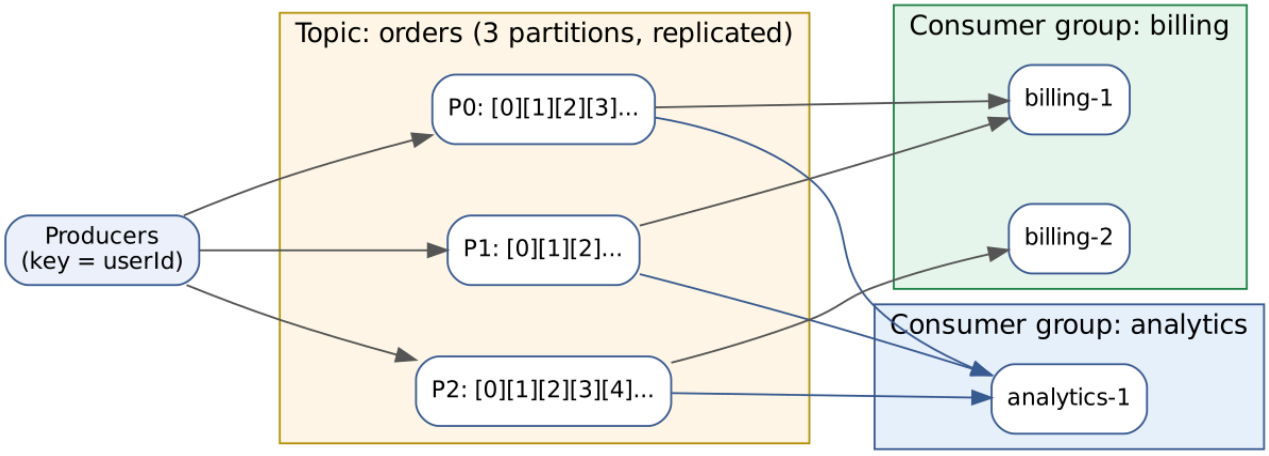
When RabbitMQ is the right pick

- Rich routing (topic patterns, headers), priorities, per-message TTL, RPC.
- Moderate throughput (tens of thousands of msgs/s per node) with low latency.
- You do **not** need to replay history; consumed messages are gone. (RabbitMQ Streams add a Kafka-like log if you do.)

These RabbitMQ samples need a running broker and `npm i amqplib`; they were not run against a live broker in this document's build.

5. Kafka: the distributed log

Kafka is not a queue that deletes messages; it is a **replicated, partitioned, append-only log** that keeps events for a retention period (days, forever, or compacted to the latest value per key). Consumers track their own position (the **offset**), so many independent consumers can read the same data, at different speeds, and can **replay** history. That makes it the backbone for event streaming, audit trails, CDC, analytics pipelines and event sourcing, at millions of messages per second.



1 partition -> max 1 consumer per group. Each group tracks its own offsets.

Concepts

Concept	Meaning
Topic	Named stream of events (`orders`)
Partition	An ordered, append-only shard of a topic. **The unit of parallelism and of ordering**
Offset	Position of a record within a partition (0, 1, 2, ...)

Concept	Meaning
Key	Hashed to choose the partition; same key means same partition means ordered
Consumer group	Consumers sharing a `groupId`; each partition is read by exactly one member of the group. Different groups are fully independent
Replication	Each partition has a leader and followers; in-sync replicas (ISR) take over if the leader dies
`acks=all` + `min.insync.replicas=2`	Producer waits until at least 2 replicas have the write: durability over latency
Retention / compaction	Delete by age or size, or keep only the newest record per key
Rebalance	When members join or leave a group, partitions are reassigned (brief pause; cooperative rebalancing reduces it)
KRaft	Kafka's built-in consensus; modern clusters need no ZooKeeper

Rules of thumb

- **Max parallelism in a group = number of partitions.** 6 partitions means at most 6 active consumers per group; more sit idle. Pick the partition count with growth in mind (adding partitions later changes the key-to-partition mapping).
- **Choose keys that spread load and preserve the ordering you need** (`userId`, `orderId`). A key such as `country` creates hot partitions.
- **Commit offsets after processing** for at-least-once; make handlers idempotent.
- **Exactly-once inside Kafka** = idempotent producer + transactions + `read_committed` consumers. Side effects outside Kafka (an email, a payment) still need idempotency.
- **Monitor consumer lag** (latest offset minus committed offset); it is the Kafka equivalent of queue depth.
- Kafka vs queues: use Kafka when you need replay, many independent consumers, ordering per key or huge throughput; use a queue when you need per-message retry, priorities, delays and simple job semantics.

Code 9: a miniature Kafka you can run (no dependencies)

This models the three ideas that matter: partitions chosen by key hash, per-group committed offsets, and replay by seeking. Save as `minilog.mjs`.

```
// A miniature Kafka: partitioned append-only log + consumer groups with
committed offsets.
import { createHash } from 'node:crypto';

class MiniKafka {
  constructor(partitions = 3) {
    this.parts = Array.from({ length: partitions }, () => []);   // each
partition = append-only array
    this.committed = new Map();                                    // group ->
[next offset per partition]
  }
  partitionFor(key) {                                              // same key
=> same partition => ordered
    return createHash('md5').update(key).digest().readUInt32BE(0) %
this.parts.length;
  }
  produce(key, value) {
    const p = this.partitionFor(key);
    const offset = this.parts[p].push({ key, value }) - 1;
    return { partition: p, offset };
  }
  offsets(group) {
    if (!this.committed.has(group)) this.committed.set(group,
this.parts.map(() => 0));
    return this.committed.get(group);
  }
  // A consumer owns some partitions; it reads from the group's committed
offset.
  poll(group, partitions, max = 10) {
    const out = [];
    for (const p of partitions) {
      const from = this.offsets(group)[p];
      this.parts[p].slice(from, from + max).forEach((rec, i) => out.push({
partition: p, offset: from + i, ...rec }));
    }
    return out;
  }
  commit(group, partition, nextOffset) { this.offsets(group)[partition] =
nextOffset; }
  seek(group, partition, offset) { this.offsets(group)[partition] = offset; }
// replay!
```

```
}
// spread partitions across the consumers of one group
const assign = (nParts, nConsumers) =>
  Array.from({ length: nConsumers }, (_, c) =>
[...Array(nParts).keys()].filter((p) => p % nConsumers === c));

const k = new MiniKafka(3);
const users = ['alice', 'bob', 'carol', 'dave', 'erin', 'frank'];
for (let i = 1; i <= 2; i++) for (const user of users) k.produce(user,
`${user}-${i}`);

console.log('partition sizes:', k.parts.map((p) => p.length));

// Group "billing": 2 consumers share 3 partitions
const billing = assign(3, 2);
billing.forEach((parts, i) => {
  const recs = k.poll('billing', parts);
  console.log(`billing-${i + 1} owns partitions [${parts}] and read:`,
recs.map((r) => r.value).join(', '));
  for (const p of parts) k.commit('billing', p, k.parts[p].length);
});

// Group "analytics": independent offsets, reads EVERYTHING
console.log('analytics (own group) read', k.poll('analytics', [0, 1,
2]).length, 'records');

// Billing is caught up now:
console.log('billing new records:', k.poll('billing', [0, 1, 2]).length);
// Replay: rewind billing on partition 0 to offset 0
k.seek('billing', 0, 0);
console.log('after seek(0) billing re-reads:', k.poll('billing', [0]).map((r)
=> r.value).join(', '));
```

Actual output: 12 records spread as partition sizes `[ 2, 4, 6 ]` (note the uneven spread from hashing only six keys, a small taste of hot partitions); `billing-1` reads partitions 0 and 2, `billing-2` reads partition 1; `analytics` independently reads all 12; after the seek, billing re-reads `bob-1, bob-2`, and each user's events stay in order.

Code 10: real Kafka with kafkajs

```
// npm i kafkajs       (local broker: any single-node KRaft docker image)
import { Kafka } from 'kafkajs';

const kafka = new Kafka({ clientId: 'orders-app', brokers: ['localhost:9092']
});
```

```
// --- create a topic (use replicationFactor 1 on a single local broker) ---
const admin = kafka.admin();
await admin.connect();
await admin.createTopics({ topics: [{
  topic: 'orders', numPartitions: 6, replicationFactor: 3,
  configEntries: [{ name: 'min.insync.replicas', value: '2' }, { name:
'retention.ms', value: String(7 * 24 * 3600 * 1000) }],
}] });
await admin.disconnect();

// --- producer: idempotent, durable, keyed ---
const producer = kafka.producer({ idempotent: true, maxInFlightRequests: 5
});
await producer.connect();
await producer.send({
  topic: 'orders',
  acks: -1,                                              // all in-sync
replicas
  messages: [{ key: 'user-42', value: JSON.stringify({ type: 'OrderPlaced',
orderId: 1001 }) }],
});

// --- consumer: group, manual commit AFTER processing (at-least-once) ---
const consumer = kafka.consumer({ groupId: 'billing' });
await consumer.connect();
await consumer.subscribe({ topics: ['orders'], fromBeginning: true });
await consumer.run({
  autoCommit: false,
  eachMessage: async ({ topic, partition, message }) => {
    const event = JSON.parse(message.value.toString());
    await handle(event);                                    // must be
idempotent
    await consumer.commitOffsets([{ topic, partition, offset:
String(Number(message.offset) + 1) }]);
  },
});
```

Code 11: retry and dead-letter topics, plus replay

```
// On failure: park the record in a DLQ topic with context, then move on
(commit).
async function handleOrFail({ topic, partition, message }) {
  try {
    await handle(JSON.parse(message.value.toString()));
  } catch (err) {
```

```
    await producer.send({ topic: `${topic}.dlq`, messages: [{
      key: message.key,
      value: message.value,
      headers: { error: err.message, origTopic: topic, origPartition:
String(partition), origOffset: message.offset },
    }] });
  }
  await consumer.commitOffsets([{ topic, partition, offset:
String(Number(message.offset) + 1) }]);
}

// Replay: after fixing a bug, rewind a consumer group to the beginning of
the topic.
//   (the group must be stopped first)
const adm = kafka.admin();
await adm.connect();
await adm.resetOffsets({ groupId: 'billing', topic: 'orders', earliest: true
});
// or to a specific point: await adm.setOffsets({ groupId, topic, partitions:
[{ partition: 0, offset: '1500' }] });
```

Code 12: exactly-once consume-transform-produce with a transaction

```
const txProducer = kafka.producer({ transactionalId: 'enricher-1',
idempotent: true, maxInFlightRequests: 1 });
await txProducer.connect();

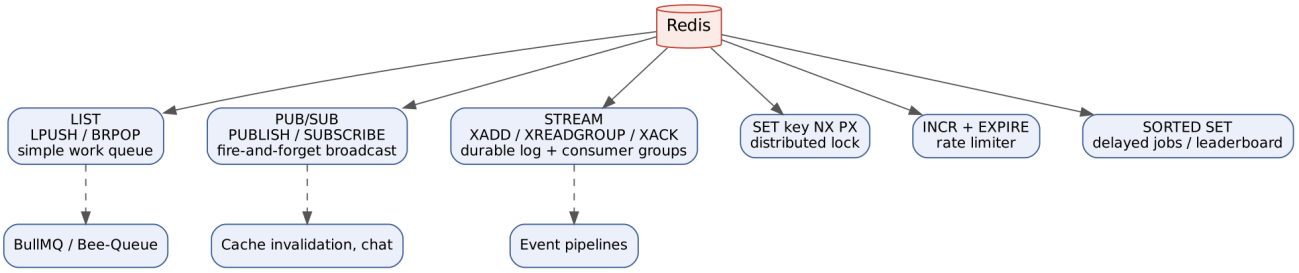
const enricher = kafka.consumer({ groupId: 'enricher', readUncommitted: false
});   // read_committed
await enricher.connect();
await enricher.subscribe({ topics: ['orders'] });
await enricher.run({
  autoCommit: false,
  eachMessage: async ({ topic, partition, message }) => {
    const tx = await txProducer.transaction();
    try {
      const enriched = { ...JSON.parse(message.value.toString()), enrichedAt:
Date.now() };
      await tx.send({ topic: 'orders-enriched', messages: [{ key:
message.key, value: JSON.stringify(enriched) }] });
      await tx.sendOffsets({                                  // output +
offset commit are ONE atomic unit
        consumerGroupId: 'enricher',
        topics: [{ topic, partitions: [{ partition, offset:
String(Number(message.offset) + 1) }] }],
      });
```

```
      await tx.commit();
    } catch (e) { await tx.abort(); throw e; }
  },
});
```

The kafkajs samples need a running broker and `npm i kafkajs`; they were not run against a live broker in this document's build. Code 9 was run.

6. Redis: the in-memory Swiss Army knife

Redis is an in-memory data-structure server that executes commands one at a time, so each command (and each Lua script) is **atomic**. That single property makes it the favourite tool for small distributed building blocks: lightweight queues, pub/sub, streams, locks, rate limiters, counters and caches.



Three messaging tools inside Redis

	List (`LPUSH` / `BLMOVE`)	Pub/Sub (`PUBLISH`)	Stream (`XADD` / `XREADGROUP`)
Model	Work queue	Broadcast	Log with consumer groups
Persistence	Yes (RDB/AOF)	**None**: offline subscribers miss messages	Yes
Ack and retry	Manual (processing list)	No	Built in (`XACK` , pending list, `XAUTOCLAIM`)
Replay	No	No	Yes (read from any ID)
Best for	Simple jobs, BullMQ	Cache invalidation, live updates, chat presence	Reliable event pipelines at modest scale

Other distributed uses

- **Distributed lock:** `SET key token NX PX ttl` acquires; a Lua script releases only if the token still matches. The TTL protects against crashed holders.
- **Rate limiting:** counter with expiry (fixed window), sorted set (sliding window) or token bucket in Lua.
- **Delayed jobs:** sorted set scored by due time; pollers claim with `ZREM`.
- **Idempotency keys:** `SET key 1 NX EX 86400` returns OK only the first time.
- **Leader election:** a lock with TTL that the leader keeps renewing.

Limits to know

- Memory-bound and **asynchronous replication**: a failover can lose the last writes, so a lock or a message can vanish in rare cases. For correctness-critical locking add a **fencing token** (monotonic counter checked by the protected resource).
- Redis Cluster splits keys across 16,384 hash slots; multi-key commands and Lua scripts need all keys in the same slot (use hash tags like `{user:42}:cart`).
- Pub/sub is at-most-once. If you cannot lose it, use Streams, or Kafka/SQS.

Verification note: Code 14 to 17 rely on the commands below, which I ran against a real Redis 7.0 server using `redis-cli`: a second `SET NX` on a held lock returned nil, releasing with the wrong token returned 0 and with the right token returned 1, the sliding-window script allowed 3 of 5 calls, a stream consumer group delivered, listed as pending, auto-claimed and acknowledged an entry, and the sorted set returned only due jobs. The JavaScript wrappers themselves need `npm i ioredis` (that package could not be installed in this build environment).

Code 13: pub/sub (use a dedicated connection for subscribing)

```
import Redis from 'ioredis';

const pub = new Redis();
const sub = new Redis();            // a subscribing connection can't run
normal commands

await sub.subscribe('cache:invalidate');
await sub.psubscribe('chat:room:*');  // pattern subscription

sub.on('message', (channel, msg) => console.log('exact', channel, msg));
sub.on('pmessage', (pattern, channel, msg) => console.log('pattern', pattern,
channel, msg));

await pub.publish('cache:invalidate', JSON.stringify({ key: 'product:42' }));
await pub.publish('chat:room:general', 'hello');
```

Code 14: reliable pipeline with Streams and consumer groups

```
import Redis from 'ioredis';
const redis = new Redis();
const STREAM = 'orders:stream', GROUP = 'billing', ME =
`worker-${process.pid}`;

// one-time setup (ignore BUSYGROUP if it already exists)
await redis.xgroup('CREATE', STREAM, GROUP, '$', 'MKSTREAM').catch((e) => {
if (!/BUSYGROUP/.test(e.message)) throw e; });

// producer: cap the stream so memory stays bounded
await redis.xadd(STREAM, 'MAXLEN', '~', 100000, '*', 'type', 'OrderPlaced',
'payload', JSON.stringify({ orderId: 1 }));

const toObj = (f) => Object.fromEntries(f.reduce((a, v, i) => (i % 2 ?
(a[a.length - 1].push(v), a) : (a.push([v]), a)), []));

async function processEntry(id, fields) {
  const evt = toObj(fields);
  await handle(JSON.parse(evt.payload));      // idempotent!
  await redis.xack(STREAM, GROUP, id);        // only now is it done
}

async function loop() {
  while (true) {
    // 1) rescue entries another worker claimed but never acked for 60s
(crashed worker)
```

```
    const [, stale] = await redis.xautoclaim(STREAM, GROUP, ME, 60_000, '0-
0', 'COUNT', 10);
    for (const [id, fields] of stale) await processEntry(id, fields);

    // 2) read new entries for this group
    const res = await redis.xreadgroup('GROUP', GROUP, ME, 'COUNT', 10,
'BLOCK', 5000, 'STREAMS', STREAM, '>');
    if (!res) continue;
    for (const [id, fields] of res[0][1]) {
      try { await processEntry(id, fields); }
      catch (e) { console.error('will be retried via XAUTOCLAIM:', id,
e.message); }
    }
  }
}
loop();
```

Code 15: distributed lock

```
import { randomUUID } from 'node:crypto';

const RELEASE = `if redis.call('get', KEYS[1]) == ARGV[1] then return
redis.call('del', KEYS[1]) else return 0 end`;

export async function withLock(redis, name, ttlMs, fn) {
  const key = `lock:${name}`, token = randomUUID();
  const ok = await redis.set(key, token, 'PX', ttlMs, 'NX');   // only one
caller gets OK
  if (ok !== 'OK') throw new Error(`busy: ${name}`);
  try {
    return await fn();                                          // keep fn
shorter than ttlMs, or renew
  } finally {
    await redis.eval(RELEASE, 1, key, token);                   // never
delete someone else's lock
  }
}

// usage: only one instance in the fleet runs the nightly invoice job
await withLock(redis, 'invoice-run', 30_000, () => runInvoices());
```

Code 16: rate limiter (sliding window, atomic in Lua)

```
const SLIDING = `
local key, now, window, limit = KEYS[1], tonumber(ARGV[1]),
tonumber(ARGV[2]), tonumber(ARGV[3])
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
if redis.call('ZCARD', key) < limit then
  redis.call('ZADD', key, now, ARGV[4]); redis.call('PEXPIRE', key, window);
return 1
end
return 0`;

export async function allow(redis, userId, limit = 100, windowMs = 60_000) {
  const ok = await redis.eval(SLIDING, 1, `rl:${userId}`, Date.now(),
windowMs, limit, randomUUID());
  return ok === 1;
}
// Express: app.use(async (req, res, next) => (await allow(redis, req.ip)) ?
next() : res.status(429).end());
```

Code 17: delayed jobs with a sorted set

```
export const schedule = (redis, jobId, runAt) => redis.zadd('delayed', runAt,
jobId);

setInterval(async () => {
  const due = await redis.zrangebyscore('delayed', 0, Date.now(), 'LIMIT', 0,
50);
  for (const jobId of due) {
    // ZREM returns 1 for exactly one poller across the whole fleet => safe
claim
    if ((await redis.zrem('delayed', jobId)) === 1) await
redis.lpush('jobs:ready', jobId);
  }
}, 1000);
```

7. Distributed scenarios and patterns

Most production incidents come from a handful of failure scenarios, and each has a standard countermeasure. This section starts with that map, then covers the patterns in depth: outbox, saga, event sourcing/CQRS, scatter-gather, consistent hashing and circuit breakers.

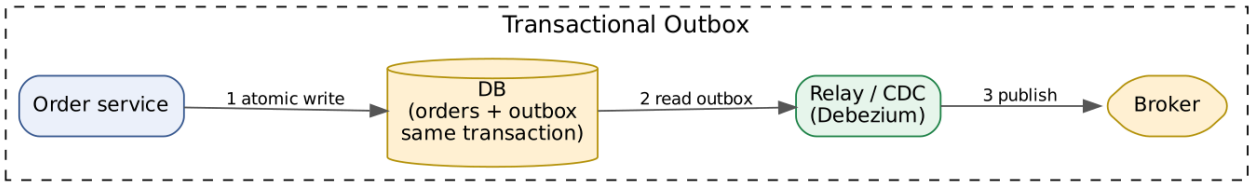
7.1 Failure scenario map

Scenario	What goes wrong	Standard countermeasure
Worker crashes mid-job	Job half done, never acked	Visibility timeout / unacked redelivery; idempotent handler
Duplicate delivery	Customer charged twice	Idempotency key + unique constraint (inbox table, `SET NX`)
Out-of-order events	"Deleted" processed before "Created"	Partition by entity key (Kafka key, SQS FIFO group); version numbers; ignore stale updates
Poison message	One bad message blocks or loops forever	Max receive count, DLQ, alert on DLQ depth
Dual write	DB commit succeeds, publish fails (or reverse)	Transactional outbox or CDC
Slow consumer / backlog	Queue grows, latency explodes	Autoscale on depth and age, prefetch limits, backpressure, drop or shed low-value work
Broker node dies	Messages unavailable	Replication (Kafka ISR, quorum queues, SQS multi-AZ), `acks=all`
Network partition	Split brain, stale reads	Quorum writes (R + W > N), leader election with fencing tokens
Hot key / hot partition	One shard overloaded	Better key (add salt/bucket), more partitions, key-splitting
Retry storm / thundering herd	Everyone retries together and finishes off the dependency	Exponential backoff with jitter, circuit breaker, rate limits
Downstream outage	Calls hang, threads pile up	Timeouts, circuit breaker, bulkheads, queue the work for later
Schema change	Old consumers crash on new fields	Backward-compatible evolution, schema registry, versioned events

7.2 Transactional outbox and inbox: solving the dual-write problem

Problem: `await db.save(order); await broker.publish(event);` can crash between the two lines, leaving a saved order with no event (or the opposite order, an event for an order that never saved). You cannot make a database and a broker commit atomically.

Fix: write the event into an **outbox table in the same database transaction** as the business data. A separate **relay** (or CDC tool such as Debezium) reads the outbox and publishes. The result is at-least-once publishing, so consumers keep an **inbox** (processed event IDs) to drop duplicates.



Code 18: outbox, relay and inbox with PostgreSQL

```
// npm i pg
import pg from 'pg';
const pool = new pg.Pool();

/* Schema:
   CREATE TABLE outbox (id uuid PRIMARY KEY, topic text, key text, payload
jsonb,
                        created_at timestamptz DEFAULT now(), published_at
timestamptz);
   CREATE TABLE inbox  (event_id uuid PRIMARY KEY, processed_at timestamptz
DEFAULT now());
*/

// 1) business write + event in ONE transaction
export async function placeOrder(order) {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    await c.query('INSERT INTO orders(id, user_id, total) VALUES ($1,$2,$3)',
[order.id, order.userId, order.total]);
    await c.query(
      `INSERT INTO outbox(id, topic, key, payload) VALUES (gen_random_uuid(),
'orders', $1, $2)`,
      [order.userId, JSON.stringify({ type: 'OrderPlaced', ...order })]);
    await c.query('COMMIT');
  } catch (e) { await c.query('ROLLBACK'); throw e; } finally { c.release();
```

```
}
}

// 2) relay: run several copies safely thanks to SKIP LOCKED
export async function relayOnce(producer) {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    const { rows } = await c.query(
      `SELECT id, topic, key, payload FROM outbox WHERE published_at IS NULL
       ORDER BY created_at LIMIT 100 FOR UPDATE SKIP LOCKED`);
    for (const r of rows) {
      await producer.send({ topic: r.topic, messages: [{ key: r.key, value:
JSON.stringify(r.payload), headers: { eventId: r.id } }] });
      await c.query('UPDATE outbox SET published_at = now() WHERE id = $1',
[r.id]);
    }
    await c.query('COMMIT');
  } catch (e) { await c.query('ROLLBACK'); throw e; } finally { c.release();
}
}

// 3) consumer: dedupe with the inbox, in the same transaction as the side
effect
export async function handleOnce(eventId, work) {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    const res = await c.query('INSERT INTO inbox(event_id) VALUES ($1) ON
CONFLICT DO NOTHING', [eventId]);
    if (res.rowCount === 0) { await c.query('ROLLBACK'); return 'duplicate';
}
    await work(c);                                      // uses the same
transaction
    await c.query('COMMIT');
    return 'processed';
  } catch (e) { await c.query('ROLLBACK'); throw e; } finally { c.release();
}
}
```

This needs PostgreSQL and `npm i pg`; it was not run in this build.

7.3 Saga: distributed transactions without two-phase commit

A business operation that spans services (order, payment, stock, shipping) cannot use one database transaction, and two-phase commit blocks and scales poorly. A **saga** splits it into local transactions, each with a **compensating action** that semantically undoes it (refund, release stock, cancel order). If step N fails, run compensations for steps N-1 down to 1.



Saga (choreography): each step has a compensating action instead of a global transaction

Style	How it works	Pros	Cons
Choreography	Services react to each other's events; nobody is in charge	Loose coupling, no central point	Flow is implicit, hard to see and debug as it grows
Orchestration	A coordinator (AWS Step Functions, Temporal, your own service) calls each step and decides what next	Explicit flow, easy to monitor, retries and timeouts in one place	Coordinator becomes a component to run and keep stateless/durable

Compensations must be idempotent and retried until they succeed; some actions cannot be undone (an email already sent), so order steps with the hardest-to-undo ones last.

Code 19: an orchestrated saga with compensation (runs as is)

```
// Saga (orchestrated): run steps in order; if one fails, run compensations
in reverse.
const log = (...a) => console.log(...a);

async function runSaga(name, steps, ctx) {
  const done = [];
  for (const step of steps) {
    try {
      log(`  -> ${step.name}`);
      await step.run(ctx);
      done.push(step);
    } catch (err) {
      log(`  !! ${step.name} failed: ${err.message}. Compensating...`);
      for (const s of done.reverse()) {
        if (s.compensate) { log(`  <- undo ${s.name}`); await
```

```
s.compensate(ctx); }
      }
      return { ok: false, failedAt: step.name };
    }
  }
  return { ok: true };
}

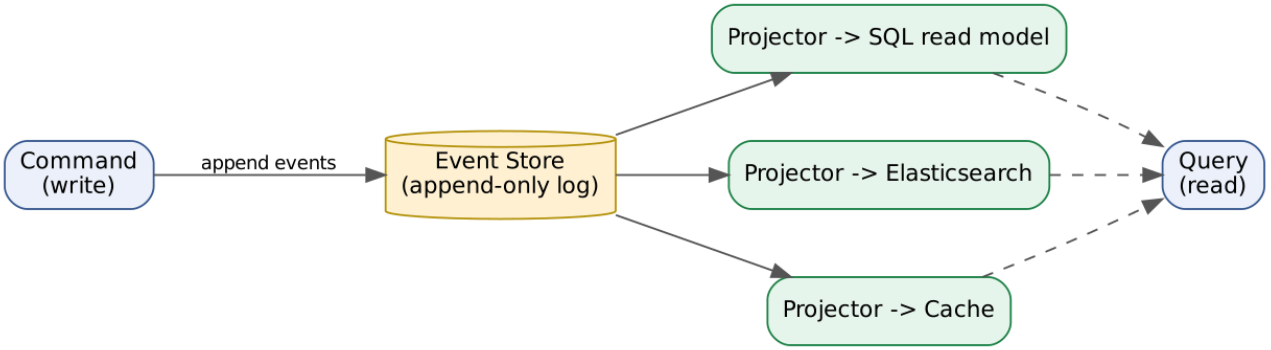
const steps = (failShipping) => [
  { name: 'create order',   run: async (c) => { c.order = 'O-1'; },
compensate: async (c) => { c.order = 'CANCELLED'; } },
  { name: 'charge payment', run: async (c) => { c.paid = true; },
compensate: async (c) => { c.paid = false; log('     refunded'); } },
  { name: 'reserve stock',  run: async (c) => { c.stock = 'reserved'; },
compensate: async (c) => { c.stock = 'released'; } },
  { name: 'book shipping',  run: async () => { if (failShipping) throw new
Error('no courier'); } },
];

log('Happy path:');   log(await runSaga('order', steps(false), {}));
log('Failure path:'); const ctx = {}; log(await runSaga('order', steps(true),
ctx), ctx);
```

Actual output on the failure path: shipping fails with "no courier", then the saga undoes stock, then payment (prints `refunded`), then the order, ending with `{ ok: false, failedAt: 'book shipping' }` and the context `{ order: 'CANCELLED', paid: false, stock: 'released' }`.

7.4 Event sourcing and CQRS

Event sourcing stores the sequence of facts (`Deposited 100`, `Withdrawn 30`) instead of the current state; the state is a fold over the events. **CQRS** separates the write model (commands validated against state) from read models (projections tailored to queries, kept up to date by consuming the events). Together they give a full audit trail, time travel and the ability to build new read models later by replaying history, at the cost of eventual consistency and more moving parts. Kafka is a natural event store.



Code 20: event sourcing and a projection (runs as is)

```
const store = [];                                   // append-only event
store
const append = (e) => store.push({ ...e, seq: store.length + 1 });

// write side: commands validate against current state, then emit events
const balance = (acct) => store.filter((e) => e.acct === acct)
  .reduce((s, e) => s + (e.type === 'Deposited' ? e.amount : -e.amount), 0);
function withdraw(acct, amount) {
  if (balance(acct) < amount) throw new Error('insufficient funds');
  append({ type: 'Withdrawn', acct, amount });
}

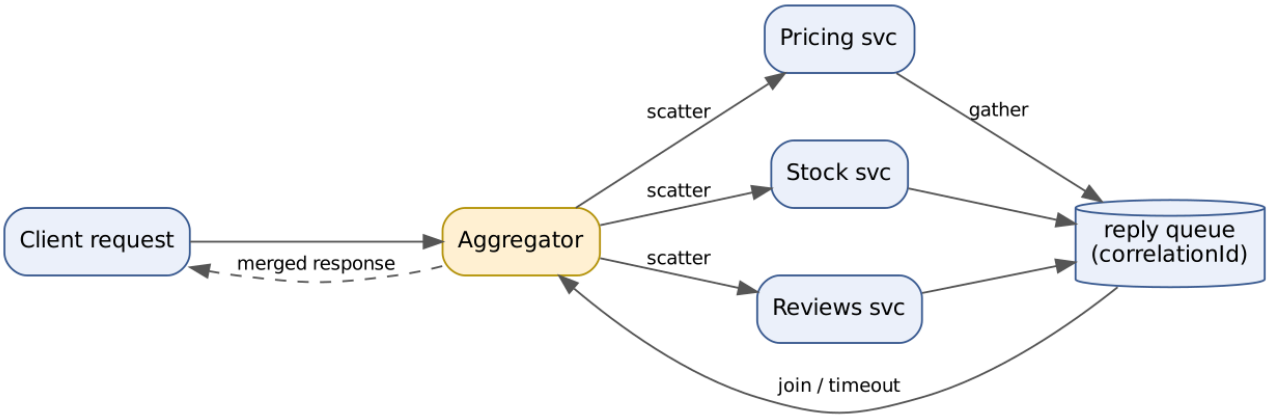
// read side: a projection built by consuming events (could live in another
service/DB)
const readModel = new Map(); let cursor = 0;
function project() {
  for (; cursor < store.length; cursor++) {
    const e = store[cursor];
    readModel.set(e.acct, (readModel.get(e.acct) ?? 0) + (e.type ===
'Deposited' ? e.amount : -e.amount));
  }
}

append({ type: 'Deposited', acct: 'A', amount: 100 });
withdraw('A', 30);
try { withdraw('A', 500); } catch (e) { console.log('rejected:', e.message);
}
project();
console.log('events:', store.map((e) =>
`${e.seq}:${e.type}:${e.amount}`).join(' | '));
console.log('read model A =', readModel.get('A'));
```

Actual output: `rejected: insufficient funds`, events `1:Deposited:100 | 2:Withdrawn:30`, read model `A = 70`.

7.5 Scatter-gather and request-reply

Fan a request out to several services in parallel, gather the replies (matching by correlation ID when using queues), and merge. The rule that matters: **set a per-call timeout and return partial results** rather than letting the slowest dependency decide your latency. Use it for search aggregation, price comparison, dashboards and pages assembled from many services.



Code 21: scatter-gather with timeouts (runs as is)

```
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const withTimeout = (p, ms) => Promise.race([p, new Promise((_, rej) =>
setTimeout(() => rej(new Error('timeout')), ms))]);

const services = {
  pricing: async () => { await sleep(30);  return { price: 19.99 }; },
  stock:   async () => { await sleep(50);  return { inStock: 12 }; },
  reviews: async () => { await sleep(900); return { stars: 4.6 }; },       //
slow dependency
};

const results = await Promise.allSettled(
  Object.entries(services).map(async ([name, fn]) => [name, await
withTimeout(fn(), 200)])
);
const merged = {}, degraded = [];
results.forEach((r, i) => r.status === 'fulfilled' ? Object.assign(merged,
r.value[1]) : degraded.push(Object.keys(services)[i]));
console.log('merged:', merged, '| degraded:', degraded);
```

Actual output: `merged: { price: 19.99, inStock: 12 } | degraded: [ 'reviews' ]`: the page renders without reviews instead of waiting 900 ms.

7.6 Sharding and consistent hashing

To spread data or load over N nodes (cache servers, database shards, Kafka partitions), you need a key-to-node mapping. `hash(key) % N` reshuffles almost everything when N changes. **Consistent hashing** places nodes (with many virtual points each) on a ring and assigns each key to the next node clockwise, so adding or removing a node moves only about 1/N of the keys. Redis Cluster uses fixed hash slots, DynamoDB and Cassandra use rings.

Code 22: consistent hash ring (runs as is)

```
import { createHash } from 'node:crypto';
const h = (s) => createHash('md5').update(s).digest().readUInt32BE(0);

class Ring {
  constructor(vnodes = 100) { this.vnodes = vnodes; this.points = []; }
  add(node)    { for (let i = 0; i < this.vnodes; i++) this.points.push({
hash: h(`${node}#${i}`), node }); this.points.sort((a, b) => a.hash -
b.hash); }
  remove(node) { this.points = this.points.filter((p) => p.node !== node); }
  get(key) {                                              // first point
clockwise from the key's hash
    const k = h(key);
    let lo = 0, hi = this.points.length;
    while (lo < hi) { const mid = (lo + hi) >> 1; this.points[mid].hash < k ?
(lo = mid + 1) : (hi = mid); }
    return this.points[lo % this.points.length].node;
  }
}
const ring = new Ring(); ['cache-1', 'cache-2', 'cache-3'].forEach((n) =>
ring.add(n));
const keys = Array.from({ length: 10000 }, (_, i) => `user:${i}`);
const before = new Map(keys.map((k) => [k, ring.get(k)]));
ring.add('cache-4');
const moved = keys.filter((k) => ring.get(k) !== before.get(k)).length;
console.log(`keys moved after adding a 4th node: ${(moved / keys.length *
100).toFixed(1)}%`);
```

Actual output: `keys moved after adding a 4th node: 26.4%`, close to the ideal 25%, versus roughly 75% with `hash % N`.

7.7 Resilience: timeouts, retries, circuit breaker, bulkhead

A slow dependency is more dangerous than a dead one, because callers pile up waiting. Always set **timeouts**; retry only idempotent calls, with backoff and jitter and a retry budget; use a **circuit breaker** to fail fast while a dependency is down; use **bulkheads** (separate pools or queues per dependency) so one bad dependency cannot consume all your capacity.

Code 23: circuit breaker (runs as is)

```
class CircuitBreaker {
  constructor(fn, { threshold = 3, cooldownMs = 200 } = {}) {
Object.assign(this, { fn, threshold, cooldownMs, fails: 0, openedAt: 0,
state: 'CLOSED' }); }
  async call(...args) {
    if (this.state === 'OPEN') {
      if (Date.now() - this.openedAt < this.cooldownMs) throw new
Error('circuit open (fast fail)');
      this.state = 'HALF_OPEN';                          // let one probe
through
    }
    try {
      const out = await this.fn(...args);
      this.fails = 0; this.state = 'CLOSED'; return out;
    } catch (e) {
      if (++this.fails >= this.threshold || this.state === 'HALF_OPEN') {
this.state = 'OPEN'; this.openedAt = Date.now(); }
      throw e;
    }
  }
}
```

A driver to exercise it:

```
let healthy = false;
const flaky = async () => { if (!healthy) throw new Error('downstream 500');
return 'ok'; };
const cb = new CircuitBreaker(flaky);
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
for (let i = 1; i <= 6; i++) { try { console.log(i, await cb.call()); } catch
(e) { console.log(i, cb.state, e.message); } }
healthy = true; await sleep(250);
console.log('after cool-down:', await cb.call(), cb.state);
```

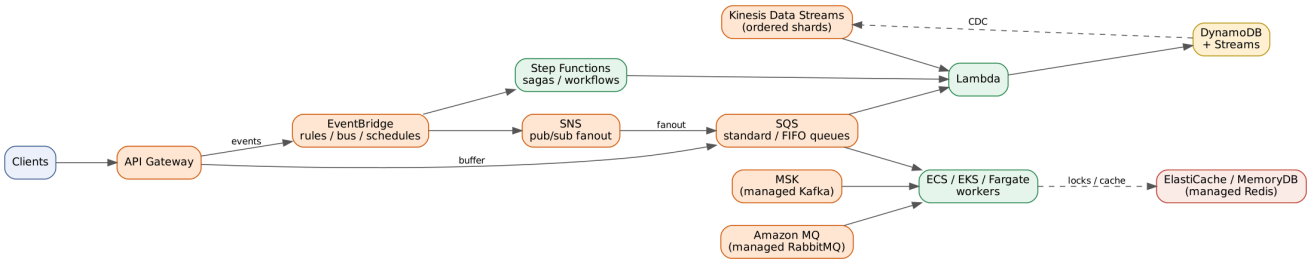
Actual output with a failing dependency: calls 1 to 3 fail normally, the breaker opens on the third failure, calls 4 to 6 fail instantly with `circuit open (fast fail)`, and after the cool-down a probe succeeds and the state returns to `CLOSED`.

7.8 Coordination, consensus and replication in brief

- **Quorum:** with N replicas, writing to W and reading from R replicas such that R + W > N guarantees a read sees the latest write (Cassandra, DynamoDB tunable consistency).
- **Consensus (Raft, Paxos):** replicas agree on one ordered log despite failures; it powers etcd, Consul, Kafka KRaft and RabbitMQ quorum queues. You rarely implement it, you choose systems that do.
- **Leader election:** exactly one instance does singleton work (schedulers, compaction). Use a consensus store (etcd, ZooKeeper), a database lock row, or a Redis lock with TTL plus a fencing token.
- **Claim-check pattern:** put the large payload in object storage (S3) and send only its reference through the broker, keeping messages small.
- **Priority and fairness:** separate queues per priority or per tenant, with weighted consumption, so one noisy tenant cannot starve the others.
- **Clocks:** do not trust wall-clock order across machines; use sequence numbers, version vectors or the broker's offsets.

8. The Amazon (AWS) toolbox

AWS offers a managed version of every pattern above, so you trade operating brokers for paying per request and accepting each service's limits. The mapping is: **SQS** is the work queue, **SNS** is pub/sub, **SNS + SQS** is the robust fanout, **EventBridge** is content-based event routing and scheduling, **Kinesis** and **MSK** are the log, **Amazon MQ** is managed RabbitMQ, **ElastiCache / MemoryDB** is managed Redis-compatible, **Step Functions** is the saga orchestrator and **Lambda** is the elastic worker.



Open-source tool to AWS service

You need	Self-hosted	AWS managed
Work queue with retries and DLQ	RabbitMQ queue, BullMQ	**SQS** (standard or FIFO)
Broadcast / fanout	RabbitMQ fanout exchange, Redis pub/sub	**SNS**, or SNS + SQS
Rule-based event routing, SaaS events, schedules	RabbitMQ topic exchange, cron	**EventBridge** (bus, rules, Pipes, Scheduler)
Replayable ordered log	Kafka	**Kinesis Data Streams** (AWS-native) or **MSK** (real Kafka)
Managed RabbitMQ	RabbitMQ	**Amazon MQ**
In-memory cache, locks, rate limits	Redis	**ElastiCache** (Valkey / Redis OSS), **MemoryDB** (durable)
Saga / workflow orchestration	Temporal, hand-rolled	**Step Functions**
Change data capture	Debezium	**DynamoDB Streams**, DMS
Workers	Your processes	**Lambda**, ECS/Fargate, EKS

Choosing among the AWS messaging services

Service	Model	Ordering	Replay	Consumers	Choose it when
SQS standard	Queue	Best effort	No	Competing	Background jobs, load levelling, near-unlimited throughput
SQS FIFO	Queue	Strict per message group	No	Competing	Per-entity order and deduplication matter (payments per account)
SNS	Pub/sub topic	Standard: none; FIFO topic: per group	No	Push to every subscriber	Fanout to queues, Lambda, HTTP, email, SMS
EventBridge	Event bus	None guaranteed	Archive and replay	Rules with up to several targets	Routing by event content, SaaS and AWS service events, schedules

Service	Model	Ordering	Replay	Consumers	Choose it when
Kinesis Data Streams	Log in shards	Per shard (partition key)	Yes, 24 hours to 365 days	Many, shared or enhanced fan-out	Real-time analytics, clickstreams, AWS-native streaming
MSK	Kafka	Per partition	Yes	Consumer groups	You need the Kafka API, ecosystem or portability
Amazon MQ	RabbitMQ / ActiveMQ	Per queue	No	AMQP/JMS clients	Lifting existing RabbitMQ or JMS apps

Facts confirmed against AWS documentation while writing this: SQS maximum payload is 1 MiB since August 2025 (previously 256 KiB); a Kinesis shard in provisioned mode takes up to 1 MB/s or 1,000 records/s of writes and serves 2 MB/s of reads; Kinesis retention is configurable from 24 hours to 365 days; Lambda's SQS batch size can reach 10,000 for standard queues but only 10 for FIFO, and `ReportBatchItemFailures` lets a function report which messages failed. Other quantities below (SQS retention 1 minute to 14 days, visibility timeout up to 12 hours, long polling up to 20 seconds, delivery delay up to 15 minutes, FIFO deduplication over 5 minutes) are from my knowledge of the AWS docs and worth re-checking against current quotas.

SQS essentials

- **Standard:** at-least-once, best-effort ordering, practically unlimited throughput. **FIFO:** ordering within a `MessageGroupId`, deduplication by `MessageDeduplicationId`, lower default throughput (raised by batching and high-throughput mode).
- **Visibility timeout** is your lease: set it longer than your processing time. For Lambda consumers, set it comfortably above the function timeout (AWS guidance commonly cited is six times, plus any batching window).
- **Use long polling** (`WaitTimeSeconds: 20`) to cut empty receives and cost.
- **DLQ:** configure a redrive policy with `maxReceiveCount`; use DLQ redrive to send repaired messages back.
- **Delete after success**, or a message reappears after the visibility timeout.

Code 24: SQS with AWS SDK v3 (send, long-poll, batch, delete, FIFO)

```
// npm i @aws-sdk/client-sqs
import { SQSClient, SendMessageCommand, SendMessageBatchCommand,
         ReceiveMessageCommand, DeleteMessageCommand, CreateQueueCommand }
from '@aws-sdk/client-sqs';
import { randomUUID } from 'node:crypto';
```

```
const sqs = new SQSClient({ region: 'ap-south-1' });
const QueueUrl = process.env.QUEUE_URL;

// --- create a queue with a DLQ (redrive) ---
await sqs.send(new CreateQueueCommand({
  QueueName: 'jobs',
  Attributes: {
    VisibilityTimeout: '60',
    ReceiveMessageWaitTimeSeconds: '20',
    RedrivePolicy: JSON.stringify({ deadLetterTargetArn: process.env.DLQ_ARN,
maxReceiveCount: 5 }),
  },
}));

// --- producer ---
await sqs.send(new SendMessageCommand({
  QueueUrl,
  MessageBody: JSON.stringify({ type: 'resize', file: 'a.png' }),
  MessageAttributes: { tenant: { DataType: 'String', StringValue: 'acme' } },
}));

// batch of up to 10 (cheaper and faster)
await sqs.send(new SendMessageBatchCommand({
  QueueUrl,
  Entries: [1, 2, 3].map((n) => ({ Id: String(n), MessageBody:
JSON.stringify({ n }) })),
}));

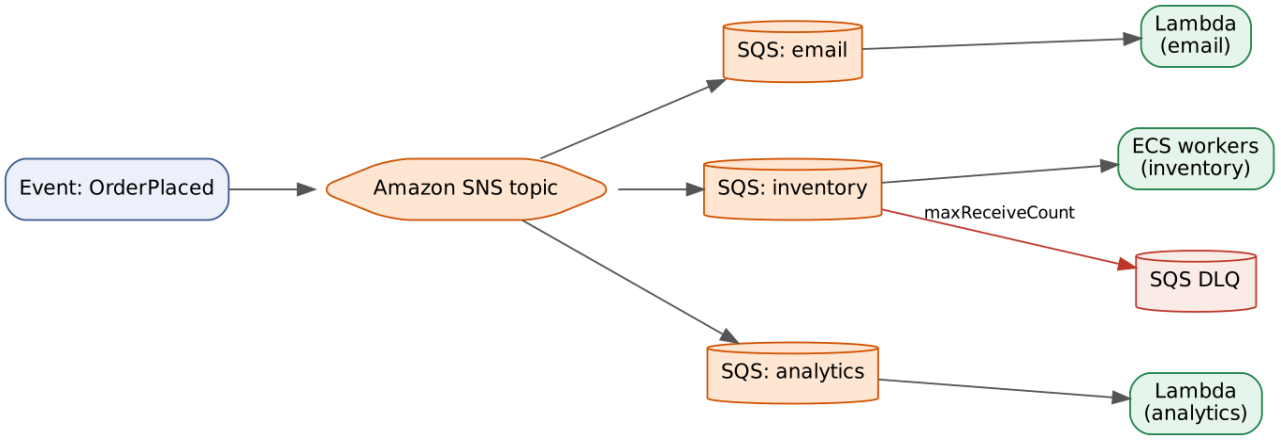
// FIFO queue (name ends in .fifo): order per group, dedupe by id
// await sqs.send(new SendMessageCommand({ QueueUrl: FIFO_URL, MessageBody:
'...',
//   MessageGroupId: `account-${accountId}`, MessageDeduplicationId:
randomUUID() }));

// --- worker loop ---
let running = true;
process.on('SIGTERM', () => { running = false; });
while (running) {
  const { Messages = [] } = await sqs.send(new ReceiveMessageCommand({
    QueueUrl, MaxNumberOfMessages: 10, WaitTimeSeconds: 20,
MessageAttributeNames: ['All'],
  }));
  await Promise.all(Messages.map(async (m) => {
    try {
      await handle(JSON.parse(m.Body));                        // idempotent
      await sqs.send(new DeleteMessageCommand({ QueueUrl, ReceiptHandle:
```

```
m.ReceiptHandle }));
    } catch (e) { console.error('left for retry/DLQ', m.MessageId,
e.message); }
  }));
}
```

SNS + SQS fanout (the standard AWS fanout)

Publish once to an SNS topic; each subscribed SQS queue receives its own copy, with its own retries and DLQ. Subscription **filter policies** let each queue take only the events it cares about.



Code 25: publish with attributes, and the Lambda consumer with partial batch failure

```
// npm i @aws-sdk/client-sns
import { SNSClient, PublishCommand } from '@aws-sdk/client-sns';
const sns = new SNSClient({ region: 'ap-south-1' });

await sns.send(new PublishCommand({
  TopicArn: process.env.TOPIC_ARN,
  Message: JSON.stringify({ orderId: 1001, total: 49.5 }),
  MessageAttributes: {
    type:   { DataType: 'String', StringValue: 'OrderPlaced' },     // used
by filter policies
    region: { DataType: 'String', StringValue: 'eu' },
  },
}));
```

```
// Lambda handler for an SQS event source (index.mjs). Enable
ReportBatchItemFailures on the mapping.
export const handler = async (event) => {
  const batchItemFailures = [];
  for (const record of event.Records) {
    try {
      const body = JSON.parse(record.body);
      await handle(body);                                  // idempotent
    } catch (err) {
      console.error('failed', record.messageId, err.message);
      batchItemFailures.push({ itemIdentifier: record.messageId });   // only
these return to the queue
    }
  }
  return { batchItemFailures };
};
```

Without partial batch responses, one bad message makes the whole batch reappear and good messages get reprocessed, so always enable it and keep handlers idempotent.

Code 26: the whole fanout as infrastructure code (AWS CDK in JavaScript)

```
// npm i aws-cdk-lib constructs
import { Stack, Duration } from 'aws-cdk-lib';
import * as sns from 'aws-cdk-lib/aws-sns';
import * as sqs from 'aws-cdk-lib/aws-sqs';
import * as subs from 'aws-cdk-lib/aws-sns-subscriptions';
import * as lambda from 'aws-cdk-lib/aws-lambda';
import { SqsEventSource } from 'aws-cdk-lib/aws-lambda-event-sources';

export class FanoutStack extends Stack {
  constructor(scope, id, props) {
    super(scope, id, props);
    const topic = new sns.Topic(this, 'OrderEvents');

    for (const name of ['email', 'inventory', 'analytics']) {
      const dlq = new sqs.Queue(this, `${name}Dlq`, { retentionPeriod:
Duration.days(14) });
      const queue = new sqs.Queue(this, `${name}Queue`, {
        visibilityTimeout: Duration.seconds(180),            // 6 x the 30 s
function timeout
        deadLetterQueue: { queue: dlq, maxReceiveCount: 5 },
      });
      topic.addSubscription(new subs.SqsSubscription(queue, {
        rawMessageDelivery: true,
```

```
        filterPolicy: { type: sns.SubscriptionFilter.stringFilter({
allowlist: ['OrderPlaced'] }) },
      }));
      const fn = new lambda.Function(this, `${name}Fn`, {
        runtime: lambda.Runtime.NODEJS_22_X, handler: 'index.handler',
        code: lambda.Code.fromAsset(`lambda/${name}`), timeout:
Duration.seconds(30),
      });
      fn.addEventSource(new SqsEventSource(queue, { batchSize: 10,
reportBatchItemFailures: true, maxConcurrency: 10 }));
    }
  }
}
```

EventBridge: route by content, schedule, replay

EventBridge matches events against JSON **rules** and sends them to targets (Lambda, SQS, Step Functions, API destinations and more). It also provides schedules (cron replacement), an event archive with replay, and Pipes to connect a source to a target with filtering and enrichment. Prefer it when producers should not know who listens, or when events come from AWS services and SaaS partners.

```
// npm i @aws-sdk/client-eventbridge
import { EventBridgeClient, PutEventsCommand } from '@aws-sdk/client-
eventbridge';
const eb = new EventBridgeClient({ region: 'ap-south-1' });

await eb.send(new PutEventsCommand({ Entries: [{
  EventBusName: 'orders-bus',
  Source: 'shop.orders',
  DetailType: 'OrderPlaced',
  Detail: JSON.stringify({ orderId: 1001, total: 4999, country: 'IN' }),
}] }));

// A rule on that bus matches on content, e.g. only large Indian orders:
const rulePattern = {
  source: ['shop.orders'],
  'detail-type': ['OrderPlaced'],
  detail: { country: ['IN'], total: [{ numeric: ['>', 1000] }] },
};
```

Kinesis Data Streams: the AWS-native log

A stream is split into shards; the **partition key** decides the shard and ordering is per shard. Provisioned mode scales by adding shards; on-demand mode scales automatically. Several consumers can read the same stream (use enhanced fan-out for dedicated throughput per consumer), and records stay for 24 hours up to 365 days. Lambda can consume a stream directly, one batch per shard.

```
// npm i @aws-sdk/client-kinesis
import { KinesisClient, PutRecordsCommand } from '@aws-sdk/client-kinesis';
const kin = new KinesisClient({ region: 'ap-south-1' });

await kin.send(new PutRecordsCommand({
  StreamName: 'clicks',
  Records: [{ PartitionKey: 'user-42', Data: Buffer.from(JSON.stringify({
page: '/home' })) }],
}));

// Lambda consumer for Kinesis: data arrives base64-encoded
export const handler = async (event) => {
  for (const r of event.Records) {
    const payload = JSON.parse(Buffer.from(r.kinesis.data,
'base64').toString('utf8'));
    await handle(payload, r.kinesis.sequenceNumber);
  }
};
```

Step Functions: saga orchestration on AWS

The saga from section 7 becomes a state machine: each step is a Task with `Retry` (backoff) and `Catch` that routes to compensation states. State is durable, so a crash resumes where it left off, and the console shows each execution graphically.

```
{
  "StartAt": "ChargePayment",
  "States": {
    "ChargePayment": {
      "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": { "FunctionName": "charge-payment", "Payload.$": "$" },
      "Retry": [{ "ErrorEquals": ["States.TaskFailed"], "IntervalSeconds": 2,
"MaxAttempts": 3, "BackoffRate": 2 }],
      "Catch": [{ "ErrorEquals": ["States.ALL"], "Next": "FailOrder" }],
      "Next": "ReserveStock"
    },
    "ReserveStock": {
```

```
      "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": { "FunctionName": "reserve-stock", "Payload.$": "$" },
      "Catch": [{ "ErrorEquals": ["States.ALL"], "Next": "RefundPayment" }],
      "End": true
    },
    "RefundPayment": {
      "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": { "FunctionName": "refund-payment", "Payload.$": "$" },
      "Next": "FailOrder"
    },
    "FailOrder": { "Type": "Fail", "Error": "OrderFailed", "Cause": "Saga
compensated" }
  }
}
```

Managed Kafka, RabbitMQ and Redis on AWS

The code in sections 4 to 6 works unchanged against the managed versions; only connection settings change.

- **MSK (Kafka):** same kafkajs code, with brokers from the cluster's bootstrap string and TLS plus IAM or SASL/SCRAM authentication.
- **Amazon MQ for RabbitMQ:** connect with `amqps://user:pass@b-xxxx.mq.region.amazonaws.com:5671`; the amqplib code is identical.
- **ElastiCache (Valkey/Redis OSS):** `new Redis({ host, port: 6379, tls: {} })`, or `new Redis.Cluster([...], { dnsLookup: (a, cb) => cb(null, a), redisOptions: { tls: {} } })` for cluster mode. Use **MemoryDB** when Redis must be the durable primary store.

Typical AWS reference architectures

1. **Async API:** API Gateway or ALB, then SQS, then Lambda or Fargate workers, with a DLQ and a CloudWatch alarm on queue age.
2. **Event fanout:** service publishes to SNS (or EventBridge), subscribers each own an SQS queue and DLQ.
3. **Streaming analytics:** producers to Kinesis (or MSK), Lambda or Flink for processing, Firehose to S3 for the lake.
4. **Sagas:** Step Functions orchestrating Lambdas, with compensations on `Catch`.
5. **Outbox/CDC:** DynamoDB Streams or RDS plus DMS to Kinesis/EventBridge for reliable event publication.

Sources: SQS 1 MiB payload announcement (August 2025), Kinesis Data Streams quotas and limits, Lambda parameters for SQS event source mappings. The AWS SDK, CDK and Step Functions samples were not deployed to a live account in this build.

9. Choosing the right tool, cheat sheet and roadmap

Pick by the question "what happens to a message after it is consumed, and who needs to read it?": deleted after one worker (queue), copied to every subscriber (pub/sub), or kept for anyone to read again (log).

Which tool for which scenario

Scenario	Pick	Why
Background jobs in a Node app that already has Redis	**BullMQ**	Retries, delays, priorities, rate limits with little setup
Microservice events with filtered routing, RPC, priorities	**RabbitMQ**	Exchanges and bindings give flexible routing; low latency
Event streaming, replay, many independent consumers, very high throughput	**Kafka** (or MSK)	Durable partitioned log, offsets, consumer groups
Live notifications, cache invalidation, presence	**Redis pub/sub**	Fastest and simplest, loss is acceptable
Lightweight durable stream with groups, modest scale	**Redis Streams**	Ack and claim without running Kafka
Serverless background jobs on AWS	**SQS + Lambda**	No servers, built-in DLQ, scales with the queue
Fanout on AWS	**SNS + SQS**	Each subscriber isolated with its own buffer and DLQ
Route by event content, SaaS events, schedules on AWS	**EventBridge**	Rules, archive, replay, Scheduler
Long multi-service transaction with undo	**Step Functions** (or Temporal)	Durable orchestration, retries and compensation
Per-entity ordering	Kafka key or SQS FIFO message group	Order only where you need it

Side by side

	Redis	RabbitMQ	Kafka	SQS / SNS
Model	List, pub/sub, stream	Exchange, queue	Partitioned log	Queue / topic
After consume	Deleted (list), none (pub/sub), kept (stream)	Deleted	Kept until retention	Deleted
Replay	Streams only	No (Streams add it)	Yes	No
Ordering	Per list / stream	Per queue	Per partition	FIFO per group
Throughput	High, memory-bound	Tens of thousands/s per node	Millions/s per cluster	Practically unlimited (standard)
Operations	Easy	Moderate	Heaviest (managed helps)	None
Delivery	At-most-once (pub/sub); at-least-once with care	At-least-once	At-least-once; exactly-once within Kafka	At-least-once (standard); dedupe window (FIFO)

Ten questions to ask before building any async flow

1. What happens if the consumer crashes after doing the work but before the ack? (Idempotency.)
2. What happens if the message is delivered twice, or arrives out of order?
3. What is the retry policy, and where do messages go after the last retry? (Backoff, DLQ, alert.)
4. What is the ordering requirement, and what key preserves it?
5. What happens when producers outpace consumers? (Backpressure, autoscaling, shedding.)
6. Do I need to replay history? (If yes, a log.)
7. How do I update my database and publish an event atomically? (Outbox.)
8. If step 3 of 5 fails, how do I undo steps 1 and 2? (Saga.)
9. How do I see what is happening? (Queue depth, message age, consumer lag, DLQ depth, correlation IDs, tracing.)

10. How will the message schema evolve without breaking old consumers?

Zero-to-hero roadmap

1. **Foundations (days 1 to 2):** read section 1; run `queue.mjs` and `fanout.mjs`; explain ack, visibility timeout, at-least-once and idempotency to a friend.
2. **Queues on Redis (days 3 to 4):** run Redis in Docker; build an email-sending worker with BullMQ with retries, delayed jobs and a graceful shutdown.
3. **RabbitMQ (days 5 to 7):** build a topic-exchange event bus with three services, a DLX and a delayed-retry loop; add publisher confirms.
4. **Kafka (days 8 to 12):** run `minilog.mjs`, then a single-node Kafka; produce keyed events, run two consumer groups, kill a consumer to see a rebalance, reset offsets to replay, watch lag.
5. **Patterns (days 13 to 16):** implement the outbox with Postgres, an orchestrated saga with compensation, a circuit breaker around a flaky call, a sliding-window rate limiter.
6. **AWS (days 17 to 21):** deploy the SNS to SQS to Lambda fanout (section 8, Code 26) with a DLQ and alarms; then a Step Functions saga; then a Kinesis stream with a Lambda consumer.
7. **Hero level:** run a failure drill (kill workers, pause the broker, inject duplicates and slow consumers) and write down what broke; read the Kafka, RabbitMQ and AWS documentation on delivery guarantees; study "Designing Data-Intensive Applications" (Kleppmann) for replication, partitioning and consensus.

How to run the examples

The in-memory samples (Code 1, 4, 9, 19, 20, 21, 22, 23) need only Node 18 or newer: save each as a `.mjs` file and run `node file.mjs`. For the broker samples: `docker run -p 6379:6379 redis`, `docker run -p 5672:5672 -p 15672:15672 rabbitmq:3-management`, and a single-node Kafka image such as `apache/kafka`, plus the matching npm package (`ioredis`, `bullmq`, `amqplib`, `kafkajs`). For AWS, use an account with least-privilege IAM roles and set a billing alarm before experimenting.

Diagrams

All diagrams were generated with Graphviz from DOT sources (`dot -Tpng file.dot -o file.png`); the sources are included in the download bundle sent with this document.